

Faculdade de Informática e Administração Paulista — FIAP
Pós-Graduação IA para DEVs

Fase 5 — Privacidade, Segurança de Dados e Aplicações Práticas

RELATÓRIO TÉCNICO

Hackathon - STRIDE Architecture Analyzer

Gustavo Paulino de Lima Neto

gustavopln@gmail.com

Repositório (GitHub): github.com/gustavopln/hackaton-fiap-stride-ai

Vídeo de apresentação (YouTube): <https://youtu.be/IYqyNsSzGK0>

README do projeto: <https://github.com/gustavopln/hackaton-fiap-stride-ai/blob/main/README.md>

Este documento é o entregável técnico do projeto, descreve o problema, a arquitetura da solução, as decisões técnicas tomadas (e por quê), os resultados obtidos e as métricas de avaliação.

Julho de 2026

Sumário

1. Introdução e Contexto	3
2. Objetivo do Projeto	3
3. Arquitetura da Solução	4
Stack técnica	4
4. Camada 1 — Detecção de Componentes (SLM / YOLOv8)	4
Dataset de treino e correção de desbalanceamento	5
Hiperparâmetros de treino	5
Métricas de avaliação	5
O que está puxando a precision para baixo	6
Por que recall, e não precision, é a métrica prioritária aqui	6
Evidências visuais do treino	7
Heurística STRIDE + base de contramedidas	8
5. Camada 2 — Validação Multi-LLM (Claude + GPT-4o)	8
Achado técnico: custo e truncamento do Claude	8
Robustez do parsing	9
6. Camada 3 — Consensus Engine	9
7. Fluxos de Dados e Travessias de Fronteira	9
Por que YOLOv8 para componentes e YOLO11-pose para fluxos?	10
8. Geração do Relatório e Interface	10
Interface (Streamlit)	11
Resumo executivo	11
Fronteiras de confiança identificadas no diagrama	11
Fluxos de dados entre componentes	11
Ameaças por componente	12
9. Resultados e Validação	12
Validação manual dos componentes e ameaças	12
Limitações conhecidas	12
10. Decisões de Arquitetura e Pivots	13
RAG (LangChain + ChromaDB) → base de conhecimento estática	13
Florence-2 → YOLOv8	13
11. Conclusão e Próximos Passos	13
Referências	14

1. Introdução e Contexto

Modelagem de ameaças é a prática de identificar, de forma sistemática, os riscos de segurança presentes numa arquitetura de software antes (ou depois) dela ser construída. O framework STRIDE — criado pela Microsoft — organiza essa análise em 6 categorias, aplicadas a cada componente/ativo do sistema:

- **Spoofing** (falsificação de identidade) — alguém se passa por outro usuário ou sistema.
- **Tampering** (violação de integridade) — modificação não autorizada de dados ou código.
- **Repudiation** (repúdio) — o usuário nega ter feito uma ação e não há como provar o contrário.
- **Information Disclosure** (divulgação de informações) — exposição de dados sensíveis a quem não deveria ver.
- **Denial of Service** (negação de serviço) — indisponibilizar o sistema para usuários legítimos.
- **Elevation of Privilege** (elevação de privilégio) — ganhar permissões além das que deveria ter.

Na prática, esse processo hoje é majoritariamente manual: um especialista em segurança olha o diagrama de arquitetura, identifica cada componente, e para cada um raciocina sobre quais das 6 categorias se aplicam, qual a severidade e quais contramedidas recomendar. É um trabalho que exige experiência, é demorado, e a qualidade do resultado varia bastante dependendo de quem faz.

Este projeto foi desenvolvido para o Hackathon da Fase 5 (Privacidade, Segurança de Dados e Aplicações Práticas) da pós-tech FIAP, com a proposta de automatizar boa parte desse processo: dado um diagrama de arquitetura cloud (PNG/JPG), o sistema identifica os componentes, aplica a análise STRIDE, e entrega um relatório completo em PDF — em minutos, não em horas.

2. Objetivo do Projeto

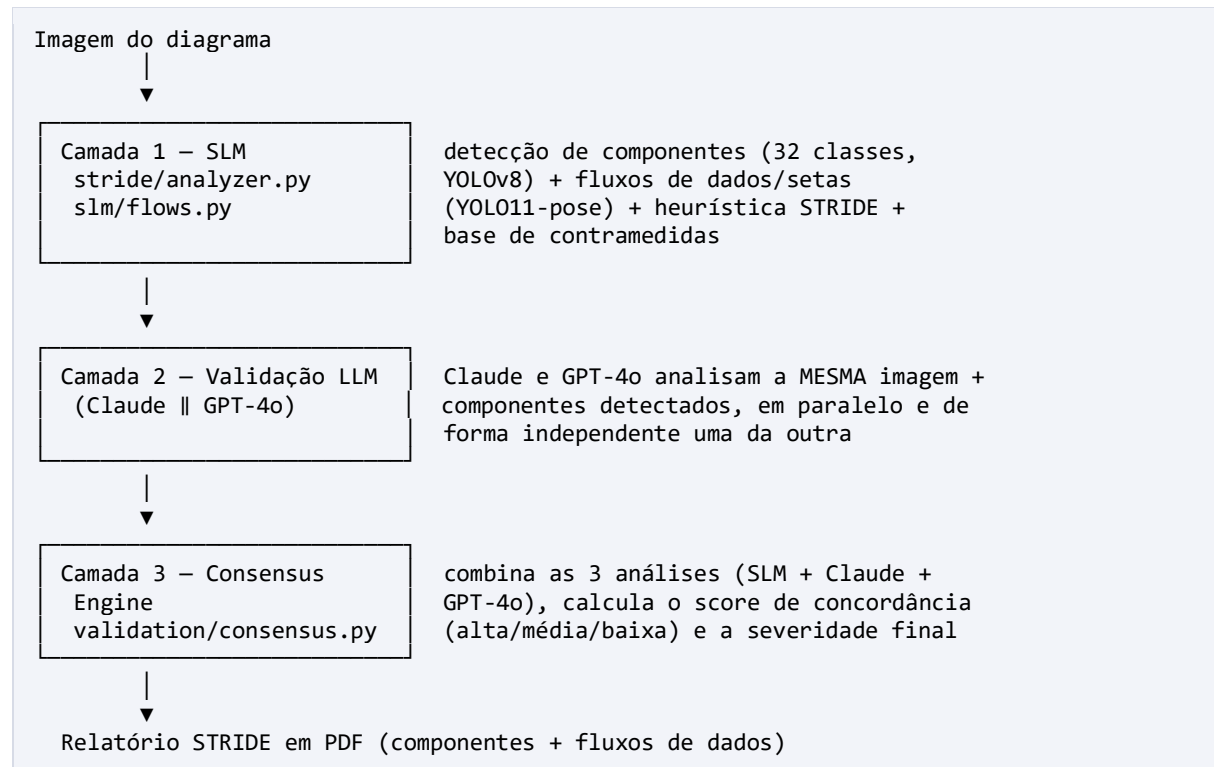
Construir um pipeline que, a partir de uma imagem de diagrama de arquitetura, produza automaticamente:

- A lista de componentes de arquitetura presentes no diagrama (via visão computacional).
- Para cada componente, quais das 6 categorias STRIDE são aplicáveis, com severidade e descrição.
- Um score de concordância por ameaça, baseado em 3 fontes de análise independentes — para que o resultado não dependa cegamente de uma única opinião (humana ou de IA).
- Contramedidas recomendadas para cada ameaça aplicável.
- Um relatório final em PDF, pronto para ser usado como ponto de partida numa revisão de segurança real.

Um objetivo não-funcional igualmente importante: o sistema precisa ser transparente sobre sua própria incerteza. Em vez de apresentar todas as ameaças com o mesmo peso, ele sinaliza explicitamente quando só uma fonte identificou algo ("baixa concordância") — para que uma ameaça duvidosa não seja tratada com a mesma confiança que uma unanimemente confirmada pelas 3 fontes.

3. Arquitetura da Solução

A solução é um pipeline de 3 camadas independentes. Cada camada analisa o mesmo diagrama à sua maneira, sem que uma valide ou dependa do resultado da outra — só na 3ª camada os 3 resultados são comparados e combinados. Essa independência é o que torna o "score de concordância" um sinal de confiança real, e não uma métrica inflada artificialmente.



Stack técnica

Camada	Tecnologias
Detecção de componentes	Ultralytics YOLOv8 (fine-tuned), PyTorch
Detecção de fluxos (setas)	YOLO11-pose — modelo publicado pelo autor do dataset (keypoints origem/destino)
Validação multi-LLM	Anthropic Claude (Messages API), OpenAI GPT-4o (Chat Completions API)
Backend / orquestração	Python, FastAPI, Uvicorn
Interface	Streamlit
Relatório	Jinja2 (templates HTML) + WeasyPrint (HTML → PDF)
Geração de dataset sintético	Biblioteca `diagrams` + Graphviz + Pillow

4. Camada 1 — Detecção de Componentes (SLM / YOLOv8)

Um modelo YOLOv8s foi treinado (fine-tuning a partir dos pesos pré-treinados no COCO) para reconhecer 32 classes de componentes de arquitetura cloud — atores, serviços de borda (CDN/WAF/gateway), computação, dados, segurança, observabilidade, integrações externas e fronteiras de confiança (VPC/subnet/autoscaling group).

Dataset de treino e correção de desbalanceamento

O dataset principal é o público guilherms/stride-architecture-components-v1, do autor Guilherme Santos (Hugging Face Datasets, licença MIT) — 4.190 imagens de diagramas AWS e Azure anotadas em 32 classes, já divididas em splits de treino (2.980 imagens), validação (840) e teste (230).

A análise exploratória desse dataset, antes do treino, revelou um desbalanceamento importante: duas classes (actor_admin e integration_messaging) tinham ZERO exemplos — nem no treino, nem na validação — o que significa que, independentemente do número de épocas, o modelo jamais as aprenderia. Outras sete classes tinham exemplos no treino, mas nenhum na validação (impossível medir o aprendizado), e havia extremos de frequência como compute_service (1.170 exemplos) contra compute_worker (apenas 30).

A correção foi gerar diagramas sintéticos com a biblioteca `diagrams` + Graphviz (slm/generate_dataset.py), com bounding boxes calculadas automaticamente a partir das coordenadas do Graphviz, reforçando exatamente as classes deficitárias. Foram agregadas 646 imagens sintéticas ao dataset real — 516 no split de treino e 130 no de validação — fechando em 3.496 imagens de treino, 970 de validação e 230 de teste (o split de teste foi mantido 100% real, sem contaminação sintética, para a avaliação final). A matriz de confusão do modelo final confirma que a correção funcionou: a diagonal está forte para praticamente todas as 32 classes, incluindo as duas que antes não tinham nenhum exemplo.

O treino foi executado no Kaggle (GPU T4), com o notebook versionado no repositório (slm/notebook-stride-architecture-components.ipynb) incluindo os outputs reais de execução — da análise do desbalanceamento à mescla sintética e à avaliação final.

Hiperparâmetros de treino

Parâmetro	Valor
Modelo base	yolov8s.pt (pré-treinado, COCO)
Épocas	100 (parada antecipada na época 63 — patience=20; melhor checkpoint na época 43)
Tamanho de imagem	640×640
Batch size	16
Hardware	Kaggle — 1 GPU T4

Métricas de avaliação

Métricas do melhor checkpoint (época 43, early stopping na época 63/100 com patience=20), avaliadas no split de validação:

Métrica	Valor	Leitura
mAP50	0,72	Precisão média com sobreposição $\geq 50\%$ entre caixa prevista e real — a métrica primária para este caso de uso (identificar QUAL componente existe, sem exigir caixa pixel-perfeita)
mAP50-95	0,59	Média em 10 limiares de sobreposição cada vez mais rígidos (50% a 95%) — medida complementar de precisão geométrica da caixa
Precision	0,54	Moderada — reflete confusão entre classes visualmente parecidas (ver argumento de design abaixo)
Recall	0,89	Alto — só ~10% dos componentes presentes passam sem detecção

O modelo também foi avaliado no split de teste — 230 imagens 100% reais, nunca vistas durante o treino nem usadas no early stopping — obtendo mAP50 de 0,82 e mAP50-95 de 0,65, consistente com a validação. Avaliar em um conjunto totalmente isento dá mais confiança de que as métricas não estão infladas por ajuste ao próprio conjunto de validação.

A escolha consciente foi priorizar recall sobre precision — a Camada 1 é um estágio de triagem, não a decisão final. Como em um exame de triagem médica (onde um falso negativo é um paciente doente liberado, e um falso positivo é apenas um exame confirmatório a mais), o custo de um componente não detectado — que nunca receberá análise STRIDE — é estruturalmente pior que o de uma detecção espúria, que Claude e GPT-4o têm a chance de descartar na Camada 2 ao reavaliar a imagem original com mais contexto. A classe individualmente mais fraca é `actor_admin`, que se confunde parcialmente com `actor_user` (ícones de pessoa visualmente similares) — limitação documentada em vez de omitida. Gráficos completos (matriz de confusão, curvas P/R/F1) estão versionados em `docs/training_results/`.

O que está puxando a precision para baixo

A precision agregada de 0,54 não é uniforme entre as 32 classes — o log de validação por classe (val, melhor checkpoint) mostra um padrão claro: ela é derrubada por um grupo específico de classes raras, enquanto as classes bem representadas no dataset têm precision consistentemente acima de 0,70:

Classe	Instâncias (val)	Precision	Recall
<code>actor_admin</code>	13	0,26	0,97
<code>integration_messaging</code>	22	0,27	0,86
<code>edge_portal</code>	10	0,38	0,90
<code>compute_worker</code>	28	0,40	0,96
<code>external_entry_point</code>	9	0,43	1,00

O padrão é consistente: são justamente as classes mais raras do dataset — e, no caso de `actor_admin` e `integration_messaging`, exatamente as duas que tinham ZERO exemplos reais antes da mescla sintética (seção "Dataset de treino e correção de desbalanceamento"). Recall alto (0,86 a 1,00) mostra que o modelo aprendeu a reconhecer a forma dessas classes; precision baixa mostra que ele ainda as confunde com outras visualmente parecidas — esperado quando poucos exemplos reais ensinam a forma geral, mas não o suficiente da variação fina que separa uma classe da vizinha. Classes bem representadas — `actor_user`, `compute_service`, `data_database`, `edge_waf`, todas com 100+ instâncias — têm precision entre 0,70 e 0,76, confirmando a correlação entre volume de exemplos e precision.

Uma exceção que vale registrar: `security_identity_provider` tem precision E recall baixos (0,36 / 0,57) mesmo com 31 instâncias — não é só um efeito de amostra pequena, é uma classe genuinamente mais difícil de distinguir visualmente (provável confusão com outros ícones de segurança/identidade), uma limitação real do modelo documentada em vez de escondida.

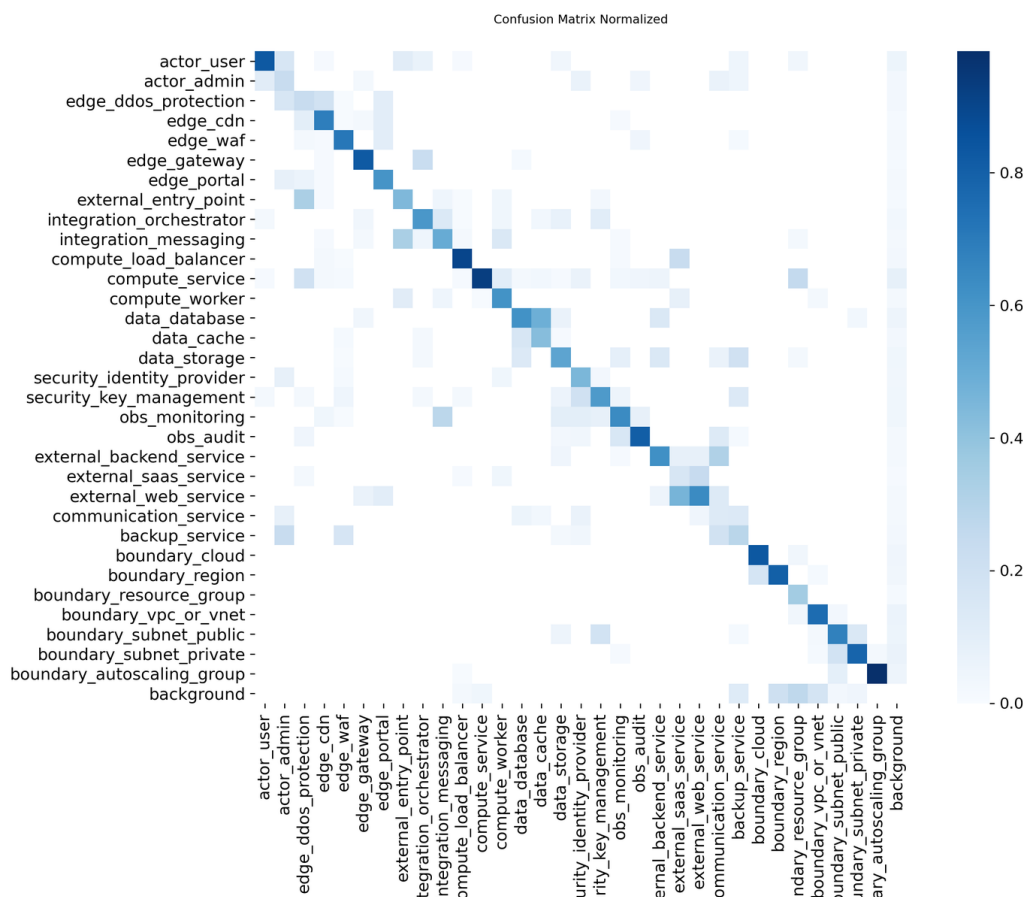
Por que recall, e não precision, é a métrica prioritária aqui

Qualquer detector reduz seus acertos e erros a 4 desfechos possíveis: verdadeiro positivo (apontou certo), falso positivo (apontou algo que não existia — um alarme falso), falso negativo (deixou passar algo que existia) e verdadeiro negativo. Precision e recall fazem perguntas diferentes sobre esses 4 números: precision mede a confiabilidade do alarme (das vezes que o modelo apontou algo, quantas estavam certas), e recall mede a cobertura (de tudo que existia para ser encontrado, quanto foi encontrado). Não existe métrica "melhor" em absoluto — existe métrica melhor para um dado custo de erro, e a pergunta certa é sempre qual dos dois erros — o alarme falso ou o deixar passar — custa mais caro no cenário.

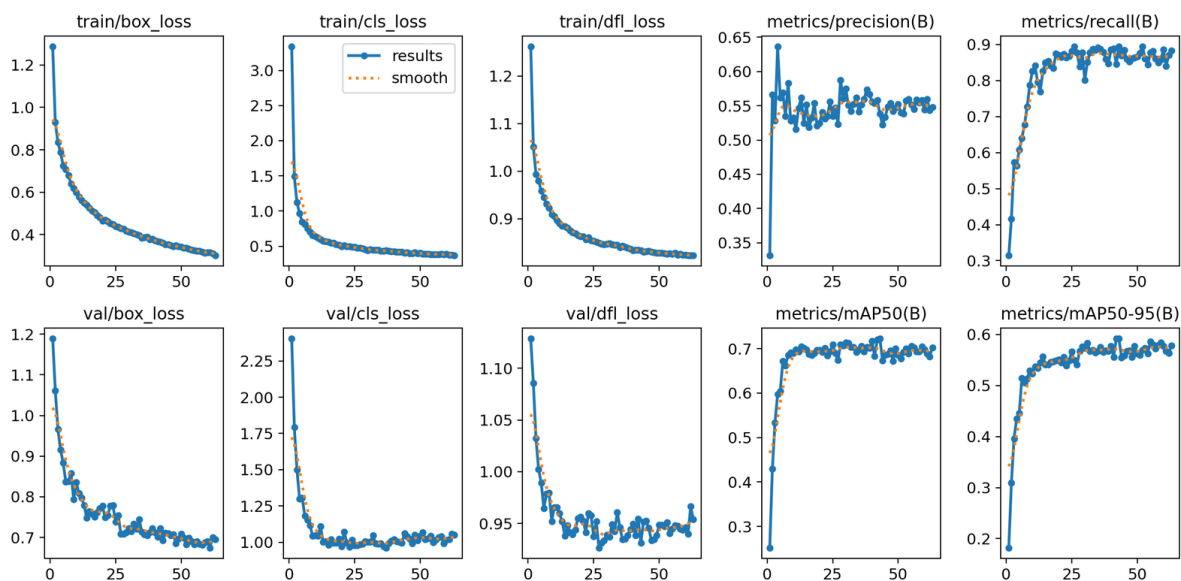
Cenário	Erro mais caro	Métrica priorizada
Triagem de problema cardíaco	Falso negativo (doente liberado)	Recall (sensibilidade)
Deteção de componentes p/ STRIDE (Camada 1)	Falso negativo (componente sem análise)	Recall
Filtro de spam	Falso positivo (e-mail legítimo perdido)	Precision
Bloqueio de cartão por fraude	Falso positivo (cliente legítimo bloqueado)	Precision

Dois ressalvas técnicas completam o raciocínio. Primeiro, por que não usar acurácia como métrica única: em dados desbalanceados ela engana — se 99% dos casos forem da classe majoritária, um modelo que sempre responde a classe majoritária tem 99% de acurácia e recall zero, nunca detectou o caso raro. Segundo, quando os dois erros custam parecido existe o F1-score (média harmônica entre precision e recall, que só fica alto se as duas estiverem altas) — não é usado como métrica de decisão neste projeto justamente porque os custos aqui são assimétricos, e reportar F1 por reflexo esconderia essa assimetria em vez de explicá-la.

Evidências visuais do treino



Matriz de confusão normalizada (32 classes + background): diagonal forte em praticamente todas as classes — incluindo `actor_admin` e `integration_messaging`, que tinham zero exemplos antes da mescla sintética. As confusões residuais concentram-se entre `actor_admin/actor_user` (ícones de pessoa visualmente similares) e no bloco de serviços externos. Arquivo completo em [docs/training_results/confusion_matrix_normalized.png](#).



Curvas de treino (63 épocas até o early stopping): as losses de treino e validação estabilizam sem divergir — overfitting leve, esperado para o tamanho do dataset, sem sinal de colapso — enquanto precision/recall/mAP sobem e estabilizam. Arquivo completo em docs/training_results/results.png.

O diretório `docs/training_results/` do repositório versiona ainda as curvas P/R/F1 por limiar de confiança, os batches de validação com gabarito vs. predição lado a lado (`val_batch*_labels.jpg` / `val_batch*_pred.jpg` — úteis para inspeção visual de erros por classe) e o `args.yaml` com a configuração completa do treino.

Heurística STRIDE + base de contramedidas

Sobre os componentes detectados, uma camada de heurística (`stride/analyzer.py`) aplica regras fixas — não IA — mapeando cada classe de componente às categorias STRIDE tipicamente relevantes, com severidades pré-definidas para combinações de alto impacto (ex. um provedor de identidade comprometido via Spoofing é "critical") e uma base de contramedidas — genéricas por categoria e específicas por classe de componente.

5. Camada 2 — Validação Multi-LLM (Claude + GPT-4o)

A saída da heurística de componentes (Camada 1, `stride/analyzer.py`) é boa, mas rígida: aplica regras fixas por classe, sem julgamento contextual sobre o diagrama como um todo — não pondera posição, papel do componente na arquitetura, nem (mesmo agora que as conexões entre componentes são detectadas pela extensão de fluxos, Seção 7) incorpora essas conexões na própria pontuação de severidade por componente: o parâmetro `'connections'` chega até essa função, mas segue reservado para uma correlação futura, não usado no cálculo hoje (ver Limitações conhecidas). Por isso, Claude e GPT-4o recebem a MESMA imagem do diagrama + a lista de componentes detectados, e cada um faz sua própria análise STRIDE olhando a imagem original com raciocínio contextual, de forma independente um do outro — nenhum vê a resposta do outro antes de responder.

Os dois modelos compartilham exatamente o mesmo prompt de sistema (mesma explicação do STRIDE, mesmo formato de saída em JSON estrito, mesma exigência de responder em português do Brasil), o que simplifica a Camada 3: ela recebe 3 fontes no mesmo formato, sem precisar tratar cada uma de um jeito diferente.

Achado técnico: custo e truncamento do Claude

Durante os testes, o Claude Sonnet 5 apresentava custo por chamada visivelmente mais alto que o GPT-4o e truncava a resposta com mais frequência. A investigação (documentação oficial da Anthropic) revelou a

causa raiz: o modelo vem com "adaptive thinking" ligado por padrão, e os tokens de raciocínio interno competem pelo MESMO orçamento de max_tokens que a resposta final — inflando custo, latência e o risco de o JSON de saída ser cortado no meio. A correção foi desligar esse modo (thinking={"type": "disabled"}), já que a tarefa aqui é extração estruturada bem especificada pelo prompt, sem necessidade de raciocínio estendido — resolveu os 3 sintomas de uma vez.

Robustez do parsing

Como a saída de um LLM não tem garantia formal de ser um JSON válido, cada validador tenta extrair o JSON da resposta (tolerando cercas de markdown), e se falhar, refaz UMA vez a chamada, reenviando a resposta malformada como contexto e pedindo explicitamente a correção — com uma mensagem mais específica quando a causa foi truncamento por limite de tokens. Se um validador falhar mesmo após a retentativa (chave ausente, erro de rede, rate limit), a falha é isolada: aquela fonte vira ausente, e a Camada 3 recalcula a concordância só com as fontes restantes, em vez de derrubar a análise inteira.

6. Camada 3 — Consensus Engine

Combina as 3 análises (SLM, Claude, GPT-4o) numa única ameaça consolidada por componente/categoria, com um score de concordância que reflete quantas fontes bateram.

Concordância entre fontes	Score
3 de 3 fontes marcam a ameaça como aplicável	Alta
2 de 3 fontes concordam	Média
Só 1 fonte identificou (ou nenhuma)	Baixa — sinalizado para revisão manual

Para severidade, quando as fontes discordam e a maior severidade envolvida é "critical", usa-se uma hierarquia de desempate (Claude > GPT-4o > SLM); fora desse caso, usa-se a média dos níveis de severidade envolvidos. As descrições de cada fonte são preservadas lado a lado no relatório final (uma linha por fonte: [SLM] / [Claude] / [OpenAI]), em vez de mescladas — o usuário pode ver o raciocínio de cada fonte, não só o resultado agregado. As contramedidas de todas as fontes são unidas e deduplicadas; quando uma ameaça é aplicável, mas nenhuma fonte trouxe uma contramedida específica, o sistema usa uma recomendação genérica por categoria STRIDE como fallback, para nunca deixar essa célula vazia.

7. Fluxos de Dados e Travessias de Fronteira

No STRIDE clássico, fluxos de dados são um dos quatro tipos de elemento analisados (junto com processos, data stores e entidades externas), tipicamente sujeitos a Tampering, Information Disclosure e Denial of Service — dados em trânsito podem ser adulterados, interceptados ou interrompidos. O sistema cobre também essa dimensão: além dos componentes (os nós do grafo da arquitetura), detecta os fluxos de dados (as arestas — as setas do diagrama).

A detecção usa o modelo YOLO11-pose publicado pelo próprio autor do dataset de componentes, treinado no dataset irmão stride-architecture-flows-v1 (as mesmas 4.190 imagens, anotando as setas em formato YOLO Pose): cada seta vira uma detecção com dois keypoints — origem e destino. Sobre essas detecções, o módulo *slm/flows.py* executa três passos: (1) casa cada ponta de seta com o componente detectado mais próximo pela Camada 1, formando conexões origem → destino — alimentando o parâmetro `connections`, reservado na arquitetura do stride/analyzer.py desde o início do projeto; (2) cruza cada conexão com as fronteiras de confiança detectadas no diagrama e sinaliza travessias (fluxo entrando na VPC, saindo de subnet privada, cruzando de subnet pública para privada — o caso prioritário); e (3) recomenda contramedidas específicas por tipo de travessia, a partir de uma base estática no mesmo padrão da base

de componentes — com um par zero trust (TLS interno e autenticação serviço-a-serviço) para fluxos que não cruzam fronteira nenhuma.

Por que YOLOv8 para componentes e YOLO11-pose para fluxos?

É uma pergunta legítima de zelo técnico, então vale deixar explícito. São duas tarefas desacopladas com formatos de anotação diferentes: a Camada 1 usa bounding boxes (um retângulo por componente, 32 classes) — o formato clássico de detecção do YOLOv8. A detecção de setas usa pose (2 keypoints por seta — origem e destino), formato que exige a variante -pose da arquitetura. O modelo de componentes foi treinado do zero para este projeto, em bounding box; o modelo de fluxos é de terceiros — publicado pelo próprio autor do dataset de fluxos — e está disponível apenas na variante YOLO11-pose, sem equivalente -pose em YOLOv8 publicado publicamente.

As duas versões não trocam dados nem competem no mesmo pipeline: cada uma roda de forma independente sobre a mesma imagem, e seus resultados só se encontram na camada de geração do relatório, por casamento de coordenadas — nunca por compatibilidade de pesos ou de arquitetura. Para efeito prático, YOLOv8 e YOLO11 são duas gerações da mesma família Ultralytics, com a mesma API de treino/inferência e o mesmo formato de pesos (.pt), diferindo principalmente em blocos internos da arquitetura — ganho incremental de velocidade/precisão, não uma mudança de paradigma. Retreinar o modelo de componentes em YOLO11 só para uniformizar a versão foi avaliado e descartado: exigiria revalidar do zero as métricas já reportadas neste documento (mAP50 0,72, recall 0,89) sem ganho concreto, já que os dois modelos nunca interagem numericamente — o caminho mais arriscado dado o prazo do hackathon, para um benefício puramente estético.

↔ **Para quem vem de PHP/JS, como eu:** Pense em dois microsserviços usando versões diferentes, mas compatíveis, do mesmo motor de banco — um roda PostgreSQL 15, outro PostgreSQL 16. Cada um fala com seu próprio banco pelo protocolo padrão, e a integração entre eles acontece só pelos dados trocados, nunca pelo mecanismo interno do banco. A diferença de versão ali não é uma inconsistência — é uma decisão de manutenção documentada.

No teste com a arquitetura AWS de referência do hackathon, 9 setas detectadas viraram 7 conexões mapeadas — incluindo a cadeia completa de borda (usuário → Shield → CloudFront → WAF/ALB, com o fluxo CloudFront → ALB sinalizado entrando na fronteira da nuvem, na região, na VPC e na subnet pública) e a replicação entre os bancos RDS cruzando de uma subnet privada para outra. Na arquitetura Azure, o grafo API Gateway → Logic Apps → serviços externos foi reconstruído com as saídas do resource group sinalizadas.

A feature é aditiva e tolerante a falha: se os pesos do modelo de pose faltarem, a detecção falhar ou nenhuma seta for mapeada, a análise STRIDE segue normalmente com um aviso explícito ao usuário — mesma filosofia de degradação parcial dos validadores LLM. Limitação conhecida: o detector cobre bem setas sólidas, mas perde parte das linhas pontilhadas finas (ex. conexões de monitoramento) — a confiança de detecção de cada fluxo é exibida no relatório para dar essa transparência (nota: essa confiança é do detector visual, semanticamente diferente do score de consenso 3-fontes das ameaças por componente — os fluxos têm uma única fonte de detecção, então não há consenso a medir).

8. Geração do Relatório e Interface

O resultado consolidado é renderizado em HTML (Jinja2) e convertido em PDF (WeasyPrint) — um relatório com resumo executivo (6 cartões: componentes analisados, ameaças identificadas, concordância alta/média/baixa, severidade alta+crítica), fronteiras de confiança identificadas no diagrama, a seção de fluxos de dados entre componentes (conexões, travessias e contramedidas por fluxo), e uma tabela de ameaças por componente (categoria, descrição por fonte, severidade, confiança, fontes que detectaram, contramedidas).

A interface (Streamlit) permite fazer upload do diagrama, visualizar os componentes detectados com as bounding boxes desenhadas sobre a imagem original, ver o resumo e a tabela de ameaças diretamente na tela, e baixar o relatório em PDF com um clique.



Interface (Streamlit)

Relatório de Modelagem de Ameaças — STRIDE

arquitetura_1_aws.jpg

Gerado em 26/07/2026 03:42

Pipeline: SLM (YOLOv8 + heurística) · Claude · GPT-4o · Consensus Engine

21 COMPONENTES ANALISADOS	98 AMEAÇAS IDENTIFICADAS	49 ALTA CONCORDÂNCIA (3/3 MODELOS)
30 MÉDIA CONCORDÂNCIA (2/3 MODELOS)	19 BAIXA CONCORDÂNCIA (1/3 — REVISAR MANUALMENTE)	34 SEVERIDADE ALTA/CRÍTICA

Resumo executivo

Fronteiras de confiança identificadas no diagrama

- boundary_subnet_private_1** (boundary_subnet_private) — Sub-rede sem rota direta à internet — validar que não há exposição acidental (NAT mal configurado, etc.).
- boundary_subnet_private_2** (boundary_subnet_private) — Sub-rede sem rota direta à internet — validar que não há exposição acidental (NAT mal configurado, etc.).
- boundary_vpc_or_vnet_1** (boundary_vpc_or_vnet) — Fronteira de rede virtual — validar peering, NSG/Security Groups e rotas entre VPCs/VNets.

Fronteiras de confiança identificadas no diagrama

Fluxos de dados entre componentes

Fluxos de dados detectados (9 setas → 7 conexões mapeadas, 2 sem correspondência)			
FLUXO (ORIGEM → DESTINO)	CONF. DETECÇÃO	TRAVESSIAS DE FRONTEIRA / OBSERVAÇÃO	CONTRAMEDIDAS RECOMENDADAS
compute_service_4 → data_database_1	93%	sai da autoscaling group (boundary_autoscaling_group_2) — validar controle de egress/destinos permitidos; sai da autoscaling group	• Instâncias novas devem herdar os mesmos controles (golden image/IaC)

Fluxos de dados entre componentes

Ameaças por componente

security_key_management_1					
6 ameaça(s) aplicável(is)					
CATEGORIA	DESCRIÇÃO	SEVERIDADE	CONFIANÇA	DETECTADO POR	CONTRAMEDIDAS RECOMENDADAS
T Tampering	[SLM] Tampering — adulteração de dados em trânsito ou em repouso [OpenAI] Chaves podem ser adulteradas.	CRITICAL	MEDIUM	SLM OPENAI	- Assinatura e verificação de integridade dos dados (HMAC/checksum) - TLS obrigatório em trânsito e criptografia em repouso - Validação e sanitização de entradas - Rotação automática de chaves e versionamento

Ameaças por componente

9. Resultados e Validação

O pipeline foi validado ponta a ponta com as duas arquiteturas de referência fornecidas no enunciado do hackathon: um diagrama AWS multi-AZ e um diagrama Azure com API Management — ambos gerando relatórios completos em PDF.

Validação manual dos componentes e ameaças

Como parte da revisão de qualidade, os componentes detectados pelo SLM foram conferidos manualmente contra os diagramas originais (nomes e quantidades) e as descrições geradas por Claude/GPT-4o foram revisadas quanto à coerência com o contexto real de cada componente (ex.: um Load Balancer recebendo ameaças de Denial of Service e Tampering condiz com seu papel na arquitetura; um banco de dados recebendo Information Disclosure e Tampering, idem). Ao todo, entre as duas arquiteturas testadas, o pipeline identificou algo próximo de 60 ameaças aplicáveis — número consistente com a quantidade de componentes de cada diagrama × as categorias STRIDE tipicamente relevantes para cada tipo de componente.

A distribuição de concordância observada nos testes mostrou a maioria das ameaças em concordância alta ou média (as 3 fontes, ou pelo menos 2, convergindo), com uma minoria em baixa concordância — exatamente o padrão esperado de um sistema de 3 avaliadores independentes: quando o sinal é forte (ex. um WAF recebendo ameaças de Spoofing/Tampering), as 3 fontes convergem; quando é mais sutil ou específico do contexto, é comum só uma fonte capturar.

Limitações conhecidas

- A precision moderada do SLM (0,54) significa que, em diagramas com ícones visualmente parecidos entre si, pode haver classificação incorreta de classe — mitigado, mas não eliminado, pela reavaliação de Claude/GPT-4o na Camada 2.
- Completeness de anotação variável no dataset de treino: uma auditoria dos labels revelou que a anotação do dataset público é parcial em boa parte das imagens — no split de treino, 680 de 1.480 labels têm 4 ou menos anotações e 140 contêm apenas classes de fronteira (nenhum componente atacável anotado); na validação, 30 de 410 imagens estão nessa última condição. O efeito é duplo: ícones presentes, mas não anotados atuam como exemplos negativos durante o treino (ensinando o modelo a ignorá-los naquele estilo visual), e as métricas reportadas são calculadas contra esses mesmos labels parciais. Em imagens sub-anotadas, o pipeline pode retornar um relatório vazio mesmo com o modelo operando exatamente como treinado — comportamento reproduzido e confirmado em teste controlado (a imagem 0bafd37a-azure_arch_19 da validação tem apenas 4 resource groups anotados, e o modelo detecta exatamente esses 4, com 94-95% de confiança).

- O sistema depende de disponibilidade das APIs da Anthropic e da OpenAI — se ambas falharem, a análise final fica baseada só na heurística do SLM, com concordância sempre baixa.
- A heurística STRIDE por componente (Camada 1, *stride/analyzer.py*) é baseada em regras fixas por classe — o parâmetro ``connections`` chega até essa (alimentado pela entensão de fluxos, Seção 7), mas ainda não é usado na pontuação de severidade/aplicabilidade por ameaça, só na análise de travessias de fronteira que roda à parte. Unificar as duas é o próximo passo.

10. Decisões de Arquitetura e Pivots

RAG (LangChain + ChromaDB) → base de conhecimento estática

O plano original usava um pipeline de RAG para recuperar contramedidas de uma base de documentos (OWASP, MITRE ATT&CK, CWE). Foi substituído por uma base estática em código, organizada por categoria STRIDE e por classe de componente. Para o escopo do hackathon, o conjunto de contramedidas relevantes é finito e bem conhecido — indexar embeddings de documentos seria infraestrutura extra sem ganho proporcional de qualidade, além de introduzir risco de recuperação ruim/alucinação.

Florence-2 → YOLOv8

A detecção de componentes começou com o Florence-2 (modelo de visão-linguagem generalista da Microsoft) e migrou para um YOLOv8 fine-tuned — mais leve e rápido para rodar em CPU durante a demo, e com controle mais direto sobre as 32 classes de interesse via fine-tuning supervisionado, em vez de depender de prompt/parsing sobre a saída de um modelo generalista.

11. Conclusão e Próximos Passos

O projeto entrega um pipeline funcional de ponta a ponta que automatiza boa parte da modelagem de ameaças STRIDE sobre diagramas de arquitetura cloud, combinando visão computacional (YOLOv8) com validação por dois LLMs independentes (Claude e GPT-4o) e um motor de consenso que quantifica a confiança de cada ameaça identificada — em vez de apresentar tudo com o mesmo peso. O parâmetro de conexões entre componentes, reservado na arquitetura desde o primeiro dia, passou a ser alimentado com dados reais, completando a cobertura STRIDE com a análise das arestas do grafo (travessias de fronteira de confiança e contramedidas específicas), não só dos nós. Essa análise roda hoje como uma camada própria, adicional à heurística por componente — que ainda não incorpora as conexões na sua pontuação de severidade, unificar as duas é a evolução natural da próxima iteração.

Possíveis evoluções futuras:

- Unificar a análise de conexões (hoje uma camada própria, Seção 7) com a heurística por componente da Camada 1 — usar a travessia de fronteira detectada para ajustar a severidade/aplicabilidade das ameaças do próprio componente, e não só gerar a seção separada de fluxos.
- Refinar a detecção de fluxos para setas pontilhadas/tracejadas finas (hoje fora do alcance do modelo de pose) — ex. fine-tuning do detector com exemplos desse estilo, ou pós-processamento morfológico das linhas.
- Aumentar o dataset de treino do SLM com mais exemplos reais (o dataset sintético hoje só complementa 2 classes) para melhorar a precision.
- Corrigir a sub-anotação do dataset via rotulagem assistida pelo próprio modelo (usar o best.pt atual para pré-anotar as caixas e revisar manualmente — uma ordem de grandeza mais rápido que anotar do zero), retreinar com os labels completos e contribuir as correções de volta ao dataset público no Hugging Face. A mesma abordagem de triagem já aplicada neste projeto: as 2 classes com zero exemplos foram corrigidas dentro do prazo (646 imagens sintéticas); a sub-anotação sistêmica, quantificada e planejada para a próxima iteração.

- Permitir que o usuário edite/valide manualmente as detecções do SLM antes de rodar as Camadas 2 e 3, corrigindo falsos positivos/negativos antes da análise LLM.
- Adicionar um 3º/4º LLM validador para robustecer ainda mais o score de concordância em cenários de discordância entre Claude e GPT-4o.

Referências

- SANTOS, Guilherme. STRIDE Architecture Threat Modeling Dataset (AWS & Azure). Hugging Face Datasets, licença MIT. huggingface.co/datasets/guillherms/stride-architecture-components-v1
- SANTOS, Guilherme. STRIDE Architecture Flows Dataset. Hugging Face Datasets, licença MIT. huggingface.co/datasets/guillherms/stride-architecture-flows-v1 — setas de fluxo das mesmas imagens base, em formato YOLO Pose (keypoints origem/destino).
- SANTOS, Guilherme. Vision Architecture Analyzer — YOLO11 Pose. Hugging Face Models, Apache 2.0. huggingface.co/guillherms/vision-architecture-analyzer-yolo11-pose — modelo usado diretamente pela detecção de fluxos deste projeto (slm/flows.py).
- SHOSTACK, Adam. Threat Modeling: Designing for Security. Wiley, 2014 — referência conceitual do framework STRIDE.
- Anthropic. Adaptive thinking — documentação oficial: platform.claude.com/docs/en/build-with-claude/adaptive-thinking
- OWASP, MITRE ATT&CK e CWE — referências conceituais usadas na curadoria da base de contramedidas.
- Ultralytics YOLOv8/YOLO11 — documentação oficial de treino e inferência.
- Kozea/WeasyPrint#2620 — incompatibilidade entre WeasyPrint 62.x e pydyf>=0.11.0: github.com/Kozea/WeasyPrint/issues/2620
- OpenAI — documentação oficial <https://developers.openai.com/api/docs>
- Repositório do projeto: github.com/gustavopln/hackaton-fiap-stride-ai